Corrigé du contrôle continu n° 1

Algorithmes distribués (M204) | Variante B | 2026–2027

Partie 1 — QCM

Q1. 8 blocs physiques. Nombre de blocs logiques : ⌈500 / 128⌉ = 4 ; avec 2 copies chacun : 4 × 2 = 8.

Q2. HDFS recopie les blocs sur d'autres nœuds. Il restaure la réplication à partir des copies restantes, si des nœuds et des copies sont disponibles.

Q3. Tolérance aux pannes. Le service continue malgré la perte d'un serveur.

Q4. La réception dépend causalement de l'envoi. L'envoi précède la réception dans la relation de causalité.

Q5. (x, 1), (y, 1), (x, 1), (x, 1). Le mapper émet une paire par occurrence ; le total est calculé plus tard.

Q6. Le Partitioner. Il affecte chaque clé à un reducer ; les valeurs d'une même clé sont dirigées vers le même reducer.

Q7. Le sharding. Des portions distinctes de la base sont réparties entre serveurs, sans copie supplémentaire.

Q8. Le diagnostic. L'absence de signal ayant déjà été repérée, l'étape suivante consiste à identifier la cause de la panne.

Q9. La montée en charge horizontale. On augmente la capacité en ajoutant des machines.

Remarque : l'en-tête indique « 1 point par question », mais les points détaillés des neuf questions totalisent 7,5 points.

Partie 2 — Lecture des schémas

Q2.1 — Trois façons de répartir les données

Situation 1 : réplication. Le bloc E est copié sur deux nœuds. But : disponibilité et tolérance aux pannes.

Situation 2 : partitionnement local. Les données sont divisées en groupes A–M et N–Z sur une seule machine. But : organiser les données et faciliter leur accès ou leur traitement.

Situation 3 : sharding (partitionnement distribué). Des morceaux différents sont placés sur des serveurs distincts. But : répartir les données et la charge, et permettre le passage à l'échelle.

Q2.2 — Trajet du fichier et job MapReduce

Le dessin numérote les cinq cases (1), (3), (4), (5), (6), alors que l'énoncé annonce (1) à (5). Les commandes ci-dessous suivent les numéros réellement visibles sur le schéma.

Hypothèses : le conteneur s'appelle **hadoop-master** ; les scripts sont dans **/home** du conteneur ; la variable **HADOOP_STREAMING_JAR** contient le chemin du JAR Hadoop Streaming. Les commandes (1) et (6) s'exécutent sur la machine locale, les autres dans le conteneur.

(1) Machine locale → conteneur

```
docker cp logs.txt hadoop-master:/home/logs.txt
```

(3) Conteneur → HDFS : créer au besoin le dossier d'entrée, puis copier le fichier.

```
hdfs dfs -mkdir -p /user/etu/entree
hdfs dfs -put /home/logs.txt /user/etu/entree/
```

(4) Lancer le job Hadoop Streaming

```
hadoop jar "$HADOOP_STREAMING_JAR" \
  -D mapreduce.job.reduces=1 \
  -files /home/map.py,/home/reduce.py \
  -mapper "python3 map.py" -reducer "python3 reduce.py" \
  -input /user/etu/entree -output /user/etu/sortie
```

(5) HDFS → conteneur puis **(6) conteneur → machine locale**

```
hdfs dfs -get /user/etu/sortie/part-00000 /home/total.txt
docker cp hadoop-master:/home/total.txt ./total.txt
```

Le dossier HDFS de sortie ne doit pas déjà exister. Un seul reducer est demandé ici pour obtenir un seul fichier part-00000 ; adapter les chemins et le nom du conteneur à l'installation.

Q2.3 — Services du cluster

① **NameNode :** gère l'espace de noms HDFS et les métadonnées, notamment les emplacements des blocs.

② **ResourceManager :** gère et alloue globalement les ressources du cluster YARN.

③ **DataNode du worker 1 :** stocke physiquement des blocs HDFS.

④ **NodeManager du worker 1 :** gère les ressources locales et les conteneurs de calcul.

⑤ **DataNode du worker 2 :** stocke physiquement des blocs HDFS.

⑥ **NodeManager du worker 2 :** gère les ressources locales et les conteneurs de calcul.

Panne du worker 1 : son DataNode devient inaccessible. Les fichiers restent lisibles si chaque bloc concerné possède au moins une copie accessible sur un autre DataNode ; HDFS recrée ensuite les copies manquantes si les ressources le permettent. Le NodeManager du worker est lui aussi indisponible pour le calcul.

Partie 4 — Commandes HDFS

Q3.1 — Six commandes (numérotation du sujet)

Les commandes sont exécutées dans le conteneur ; les chemins locaux indiqués sont relatifs au répertoire courant.

1. Créer le dossier HDFS

```
hdfs dfs -mkdir -p /user/etu/entree
```

2. Lister récursivement tout le contenu

```
hdfs dfs -ls -R /user/etu
```

3. Copier le fichier local dans HDFS

```
hdfs dfs -put notes.csv /user/etu/entree/
```

4. Récupérer le résultat HDFS dans le conteneur

```
hdfs dfs -get /user/etu/sortie/part-00000 bilan.txt
```

5. Rendre le script local exécutable

```
chmod +x count.py
```

6. Appliquer récursivement les droits 750 dans HDFS

```
hdfs dfs -chmod -R 750 /user/etu/entree
```

Pour la commande 4, le fichier de sortie doit avoir été créé au préalable par un job : il n'est pas présent dans l'état initial indiqué.

Q3.2 — Vrai ou faux

a) Faux. Sans l'option -f, **hdfs dfs -put** n'écrase pas par défaut un fichier de destination existant.

b) Faux. hdfs dfs -cut n'est pas la commande de copie HDFS vers local ; utiliser **hdfs dfs -get** ou **-copyToLocal**.

c) Faux. « chmod 644 fichier » fixe les permissions à rw-r--r-- : il ne conserve pas nécessairement les droits antérieurs. La formulation « conserver » est ambiguë ; si elle signifie « définir », la commande est correcte.

d) Faux. Sur un même cluster, les commandes équivalentes visent les mêmes données ; **hadoop dfs** est une ancienne interface dépréciée, à remplacer par **hdfs dfs**.

Partie 3 — Compréhension du Mapper et du Reducer

Q4.1 — Rôle de mapper.py

Pour chaque ligne contenant exactement six champs séparés par des tabulations, le mapper extrait la date et le montant, calcule le numéro du jour de la semaine, puis émet la paire **(jour, montant)**.

La méthode weekday() numérote les jours de 0 (lundi) à 6 (dimanche). Une date mal formée peut provoquer une erreur : le script ne la traite pas explicitement.

Q4.2 — Rôle de reducer.py

Le reducer regroupe les montants par jour de la semaine et affiche, pour chaque jour présent, son nom et **la somme totale des montants**.

Exemple : pour le lundi, des montants 100, 150 et 200 donnent Monday : 100 + 150 + 200 = **450**.

Cette logique suppose que les entrées du reducer sont regroupées par clé de jour, comme après le tri et le shuffle de Hadoop Streaming. Les jours absents ne sont pas affichés.